

# GenOS 채팅 SSE 계약

코드서빙으로 배포한 워크플로우가 GenOS 앱 채팅에 붙었을 때, 백엔드가 보내야 하는 형식을 정리하였습니다.

## 0. 프레임 형식

```
def sse(event: str, data) -> str:
    return "data: " + json.dumps({"event": event, "data": data}, ensure_ascii=False) + "\n\n"
```

```
return StreamingResponse(gen(), media_type="text/event-stream",
                          headers={"X-Accel-Buffering": "no"})
```

- 반드시 `data:` 로 시작. genportal-api 파서는 그 외 줄을 전부 버림
- 끝에 빈 줄 2개( `\n\n` )

## 1. 이벤트 목록

코드서빙이 직접 만들어 보낼 수 있는 이벤트 종류는 이하와 같습니다.

| event                        | data                    | 용도                  |
|------------------------------|-------------------------|---------------------|
| <code>token</code>           | <code>str</code>        | 답변 텍스트 조각           |
| <code>sourceDocuments</code> | <code>list[dict]</code> | 출처 목록 → 채팅창 하단 「출처」 |
| <code>action</code>          | <code>ActionData</code> | HITL UI (선택·승인 버튼)  |
| <code>error</code>           | <code>str</code>        | 에러 문구               |
| <code>end</code>             | <code>null</code>       | 스트림 끝               |

답변이 끝난 후 통상 stream으로 출력된 토큰을 누적시킨 `result`를 최종 SSE delta에 붙이지만, GenOS에서는 `result` 이벤트는 보내지 않습니다.

최종 답변은 genportal-api가 `token` 을 전부 이어붙여 알아서 만들어 준 `full_answer` 로써 `token` 을 += 로 누적인 값입니다.

## 2. 기본 응답

### 2.1 스트리밍

```
yield sse("token", "36개월 기준 금리는 ")
yield sse("token", "연 5.9% 입니다.")
yield sse("end", None)
```

## 2.2 비스트리밍 (어드민 「워크플로우 테스트」 「검증」)

`stream` 없이 호출되는 경로입니다. SSE가 아니라 JSON 봉투로 감싸서 답합니다,

```
{"code": 0, "errMsg": "success", "data": {"text": "전체 답변"}}
{"code": 1, "errMsg": "사유", "data": {"text": ""}} # 실패도 HTTP 200
```

`question == "__verify__"` 이면 LLM을 부르지 말고 즉답 -> 코드서빙에서 워크플로우 배포 -> 검증버튼에서 사용됨.

```
if body.question == "__verify__":
    return {"code": 0, "data": {"text": "verified"}}
```

## 3. 출처 문서 노출

```
yield sse("sourceDocuments", [
    {
        "pageContent": "청크 본문",
        "metadata": {
            "file_name": "사규집.pdf",
            "file_path": "/nfs-root/docs/Document-123.pdf",
            "i_page": 17,
            "doc_id": 123,
            "security_level": 0,
        },
    },
])
```

| 필드                                   | 필수 | 용도                 |
|--------------------------------------|----|--------------------|
| <code>pageContent</code>             | Y  | 출처 카드 본문           |
| <code>metadata.file_name</code>      | Y  | 문서명                |
| <code>metadata.file_path</code>      | Y  | PDF 뷰어 열기          |
| <code>metadata.i_page</code>         | Y  | 뷰어가 열 페이지          |
| <code>metadata.doc_id</code>         | 권장 | 휴지통 문서 자동 제외       |
| <code>metadata.security_level</code> | 권장 | 사용자 등급 초과 문서 자동 제외 |

genportal-api가 받은 뒤 **보안등급·삭제 여부**로 한 번 더 걸러서 프론트에 전달합니다.

## 4. 추론 과정 노출

CoT(think 태그) 를 앱채팅 설정에서 여는 태그와 닫는 태그 모두를 설정해줍니다.

```
yield sse("token", "<think>")
yield sse("token", "17 × 23 = 17×20 + 17×3 = 340 + 51")
yield sse("token", "</think>")
yield sse("token", "391입니다.")
```

| 항목     | 값                                                                             |
|--------|-------------------------------------------------------------------------------|
| 기본 태그  | <code>&lt;think&gt; ... &lt;/think&gt;</code>                                 |
| 커스텀 태그 | 채팅 서비스 설정의 <code>cot_tags</code> ( <code>[{"open_tag", "close_tag"}]</code> ) |
| 동작     | 태그 사이는 접이식 <b>추론 영역</b> , 바깥은 답변 본문                                           |

ex) 추론 모델(GLM 등)의 `reasoning_content` 를 그대로 흘리고 싶다면 직접 think 태그로 감싸서 출력하고, 감싸는 태그를 채팅 앱 설정 배포 화면에서 설정해주면 됩니다.

```
if delta.get("reasoning_content"):
    yield sse("token", f"<think>{delta['reasoning_content']}</think>")
elif delta.get("content"):
    yield sse("token", delta["content"])
```

## 5. HITL (Human In The Loop)

### 5.1 전체 흐름

```
[턴 1] question 있음
      └─ token   "「경제」로 분류했습니다."
      └─ action  선택 UI → 여기서 스트림 종료, 사용자 응답 대기
                  앱이 질문·중간상태를 저장해야 함 ★

[턴 2] question 없음 + humanInput 있음
      └─ (저장해둔 상태 복원)
      └─ token   선택된 분기로 답변
```

### 5.2 보낼 때 — `action` 이벤트

```
{"event": "action", "data": {
  "interactionId": "<uuid>",
  "elements": [ ... ]
}}
```

`interactionId` 는 `data` 와 각 `element` 안에 모두 있어야 함.

-> element 안에 없으면 프론트 `mapActionElement` 가 렌더를 거부.

#### elements 형태 예시

```
{
  "type": "single-select",
  "interactionId": "8f2c...",
  "component": {
    "type": "single-select",
    "title": "사용할 에이전트를 선택해 주세요.",
    "options": [
      {"value": "legal", "label": "법률", "desc": "사규·법령 검색"},
      {"value": "estate", "label": "부동산", "desc": "실거래 데이터 분석"}
    ]
  }
}
```

```
}
}
```

## 선택 UI (single-select)

```
element = {
  "type": "single-select",
  "interactionId": interaction_id,
  "component": {
    "type": "single-select",
    "title": "사용할 에이전트를 선택해 주세요.", # ← FE 가 여기서 질문을 읽음
    "options": [
      {"value": "legal", "label": "법률", "desc": "사규·법령 검색"},
      {"value": "estate", "label": "부동산", "desc": "실거래 데이터 분석"},
    ],
  },
}
yield sse("action", {"interactionId": interaction_id, "elements": [element]})
```

승인·거절 UI (confirm) — element 두 개를 같은 `interactionId` 로 보냅니다.

```
elements = [
  {"type": "approve-button", "interactionId": iid,
   "component": {"type": "confirm", "title": "아래 SQL 을 실행할까요?\n\nSELECT ..."}},
  {"type": "reject-button", "interactionId": iid,
   "component": {"type": "confirm", "title": ""}},
]
yield sse("action", {"interactionId": iid, "elements": elements})
```

정리하자면, 현재 Back-End에서 GenOS Front-End로 넘길때 지원하는 HITL 타입은 아래와 같습니다.

| 지원 type                                                  | 비고                                        |
|----------------------------------------------------------|-------------------------------------------|
| <code>single-select</code>                               | 라디오 선택. 「직접 입력」 칸은 FE가 자동으로 붙임 (앱이 넣지 않음) |
| <code>multi-select</code>                                | 복수 선택                                     |
| <code>approve-button</code> / <code>reject-button</code> | 버튼 글자는 FE가 「답변 진행」 / 「답변 거절」로 넣음          |

## 5.3 받을 때 — 다음 요청 body

```
{"question": "", "stream": true,
 "humanInput": {
   "type": "proceed",
   "interactionId": "<같은 uuid>",
   "action": "submit",
   "values": {"selected": ["math"], "customInput": "초등학생도 알게"}
 }}
```

취소(X 버튼)는 `{"type": "reject", "action": "cancel"}` 이고 `values` 가 없습니다.

| 필드                              | 값                                                            |
|---------------------------------|--------------------------------------------------------------|
| <code>action</code>             | <code>submit</code> (선택 완료·승인) / <code>cancel</code> (취소·거절) |
| <code>values.selected</code>    | 선택한 옵션 value 배열                                              |
| <code>values.customInput</code> | 「직접 입력」에 쓴 텍스트                                               |

5.4 HITL 회신에 대한 주의사항

턴 2(HITL에 대한 사용자 입력)에는 원래 질문이 오지 않습니다.

`question` 이 빈 문자열이고 `messages` 도 비어서 옵니다

| 턴 | 받는 body                                                            |
|---|--------------------------------------------------------------------|
| 1 | <code>{"question": "금리가 오르면...", "stream": true}</code>            |
| 2 | <code>{"question": "", "stream": true, "humanInput": {...}}</code> |

그래서 앱이 질문과 중간 상태를 직접 보관해야하고 이는 별도 개발해야합니다.

| 방법                 | 예제            | 키                                                                   |
|--------------------|---------------|---------------------------------------------------------------------|
| interactionId 로 보관 | hitl-example  | <code>_PENDING[interaction_id]</code> (single-use, 꺼낼 때 제거)         |
| 세션 ID 로 보관         | total-example | <code>x-geonos-session-id</code> → LangGraph <code>thread_id</code> |

주의사항으로는, 인메모리기반 히스토리 관리는 **파드 재시작·복제본(HA) 2개 이상이면 정상유지될 수 없기에**. 운영에서는 Redis로 옮기거나 별도 개발 DB를 활용하는 것을 추천합니다.

6. 이미지·차트 출력

별도 이벤트 없이 `token` 안에 마크다운으로 넣습니다.

```
yield sse("token", f"조회 결과입니다.\n\n[금리 비교]({presigned_url})")
```

URL은 통상적으로는, MinIO 임시 URL을 활용하는 것으로 본사 운영망에서는 활용해 왔습니다.

7. 코드서빙으로 배포하는 경우 요청으로 오는 헤더

body는 `question` / `stream` / `humanInput` 셋이 전부고, 나머지 문맥은 헤더에 담겨 옵니다.

이하 주요 헤더

| 헤더                   | 내용                                        |
|----------------------|-------------------------------------------|
| x-genos-session-id   | 채팅 세션 ID. 보통 멀티턴·HITL 상태 보관 키로 활용합니다.     |
| x-genos-user-id      | 사용자 ID                                    |
| x-genos-access-token | 그 사용자 명의 단명 JWT. GenOS 내부 API 호출 시 그대로 전달 |
| x-genos-group-id     | 그룹 ID                                     |